

Método append()

El método **append()** se utiliza para **agregar un elemento al final de una lista**.

Cuando llamás al método **append()** sobre una lista, le pasás como argumento **un solo elemento**, y ese elemento se agrega como el **último** en la lista.

```
lista_videojuegos = ["Fortnite", "Apex", "AOE II", "Valorant"]
lista_notas = [8.50, 6.10, 10, 4.2]
lista_estudiante = ["Joaquin", "Perez", 27268, True]

lista_videojuegos.append("Warzone")
lista_estudiante.append(7.66)
```

En este ejemplo **lista_videojuegos** tenía 4 elementos y con **append()** agregamos otro más, lo mismo en **lista_estudiante**. Si quisiéramos imprimir por pantalla ambas listas:

```
['Fortnite', 'Apex', 'AOE II', 'Valorant', 'Warzone']
['Joaquin', 'Perez', 27268, True, 7.66]
```

Importante:

- **append()** **no devuelve una nueva lista**, sino que **modifica la lista original**.
- **append()** solo agrega **un elemento a la vez**. Si se pasa una **lista como argumento**, esa lista se agrega **como un único elemento**, es decir, **como una sublista dentro de la lista original**. Este comportamiento es similar al de una **matriz**, donde cada elemento representa una **fila** que, a su vez, es una lista con varias **columnas** (otros elementos). Así, se forman listas anidadas, es decir, listas dentro de listas.

```
lista_estudiante.append(lista_notas)
```

Cuando agregamos una **sublista** a una lista usando el método **append()**, esa sublista se agrega como **un único elemento dentro de la lista principal**, lo que quiere decir que **los corchetes de la sublista también se incluyen como parte de la lista principal**.

```
['Joaquin', 'Perez', 27268, True, [8.5, 6.1, 10, 4.2]]
```

En este caso, **lista_estudiante** contiene:

- **Elementos simples** como nombre, apellido, ID y un valor booleano,
- **Una sublista** con las notas del estudiante, que está dentro de sus propios corchetes.

Así, la sublista **[8.5, 6.1, 10, 4.2]** se agrega tal cual a `lista_estudiante`, y no se descompone ni separa en sus elementos individuales, sino que permanece como un único elemento dentro de la lista principal.

Cuando tenemos una lista anidada, podemos acceder a la sublista y usar **`append()`** para agregarle nuevos elementos. Para ello debemos primero ingresar en el índice de la sublista y luego utilizar el método:

```
lista_estudiante[4].append(9.5)
```

De esta forma, accedemos al índice 4 que es el quinto elemento de nuestra lista y recién en ese momento llamamos al método **`append()`**, enviando por parámetro la nueva nota.

```
['Joaquin', 'Perez', 27268, True, [8.5, 6.1, 10, 4.2, 9.5]]
```

Método `remove()`

El método **`remove()`** se utiliza para **eliminar el primer elemento que coincida con el valor proporcionado** en una lista. Es importante aclarar que no se elimina por posición (como **`pop()`**), sino por **valor**.

```
lista_videojuegos.remove("AOE II")
```

El método **`remove()`** recibe como parámetro el valor del elemento que quiero eliminar de la lista, cuando encuentra la primera coincidencia lo elimina y los que seguían a él los mueve hacia la izquierda.

```
['Fortnite', 'Apex', 'Valorant', 'Warzone', 'Resident Evil 3', 'Mario Kart 8']
```

- Solo se elimina **la primera coincidencia** con el valor **'AOE II'**.
- Si hubiera más elementos iguales y quisiéramos eliminarlos también, hay que aplicar **`remove()`** nuevamente.

Si se intenta eliminar un valor que **no está presente**, se produce un **error**. Para que no falle el programa se puede validar de varias formas, un ejemplo es con un condicional:

```
elemento = "AOE II"

if elemento in lista_videojuegos:
    lista_videojuegos.remove(elemento)
else:
    print(f"{elemento} no es parte de la lista")
```

El operador **in** se utiliza para **verificar si un valor está contenido dentro de una secuencia**, como una lista, una cadena de texto (string), una tupla, un diccionario o un conjunto (set).

- Retorna **True** si el valor se encuentra dentro de la secuencia.
- Retorna **False** si el valor no está presente.